A Non-dominated Sorting Biased Random-Key Genetic Algorithm for the Multi-Objective Physical Cell Identity Assignment Problem

Luis H. P. Mendes Fábio L. Usberti Mário C. San Felice Carlos E. de Andrade

Institute of Computing – University of Campinas (UNICAMP)
Department of Computing – Federal University of São Carlos (UFSCar)
AT&T Labs Research

luis.mendes@ic.unicamp.br

16th Metaheuristics International Conference (MIC 2026)

Presentation Overview

1. Motivation & Context
2. Problem Definition
3. Proposed Method
4. Computational Experiments
5. Results & Analysis
6. Conclusion

Motivation
&
Context

Why PCI assignment is a hard (and practical) optimization task

Each Radio Access Network (RAN) cell must broadcast a **Physical Cell Identity (PCI)** so User Equipments (UEs) can detect and distinguish cells.

The PCI space is **limited** (e.g., **504** values in LTE; **1008** in 5G), so PCIs must be **reused**.

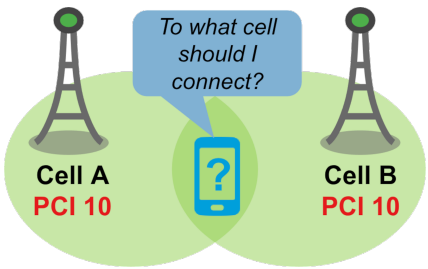
Poor reuse leads to technological conflicts (collisions, confusions, modulo-*k* interference) that degrade measurements and mobility.

Operators also care about **operational objectives**: minimize disruptive PCI changes (PCI shuffles) and avoid deployment failures during staged rollouts.

Technological constraints – Collisions

Neighboring cells must have different PCIs.
Otherwise, a **collision** occurs, and
the UE cannot connect to either cell.
For example, UE in the figure cannot connect to a cell because both cells A and B have the same PCI 10.

Also, due to constraints in the primary and second sync signal and the 5G numerology, it is very desirable that neighboring PCIs be different in modules.

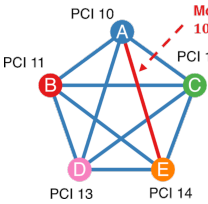


Technological constraints – Collisions

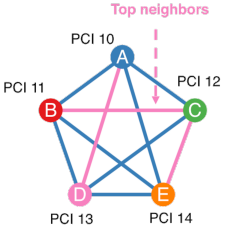
However, it is extremely hard to ensure module-3 and module-4 assignments in large production networks.

The reason is that it is very common to have a cluster of size ≥ 5 in which all cells are neighbors of the other cells with the cluster.

Precisely speaking, if we model these cells as nodes and the neighborhood relationships as edges in a neighboring graph, it is very common to find cliques of size $5+$.



Cannot find a module-4 assignment because we need, at least, 5 module-4 different colors/PCIs



Top Neighbors
A, D: $10 \not\equiv 13 \pmod 4$ OK!
B, C: $11 \not\equiv 12 \pmod 4$ OK!
C, E: $12 \not\equiv 14 \pmod 4$ OK!

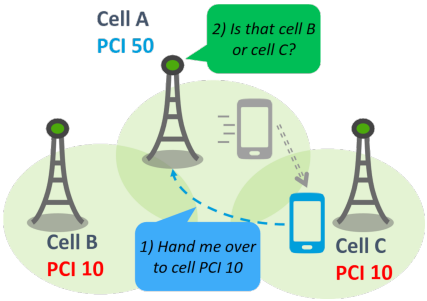
Regular neighbors
A, B: $10 \neq 11$ OK!
A, C: $10 \neq 12$ OK!
A, E: $10 \neq 14$ OK!
B, D: $11 \neq 13$ OK!
B, E: $11 \neq 14$ OK!
C, D: $12 \neq 13$ OK!
D, E: $13 \neq 14$ OK!

Technological constraints – Confusions

Confusion occurs when a cell has two neighbors with the same PCI.
When a UE asks for handover, the base station does not know to which cell it should hand the UE over.

In other words, if **second-level neighbors (neighbor of neighbors)** share a PCI, confusion may occur.

Example: The UE is moving from cell A to cell C. Then, the UE asks Cell A to hand the connections over to the cell with PCI 10.
But now, Cell A cannot determine whether UE refers to Cell B or Cell C since they have the same PCI.



Operational constraints – PCI shuffle

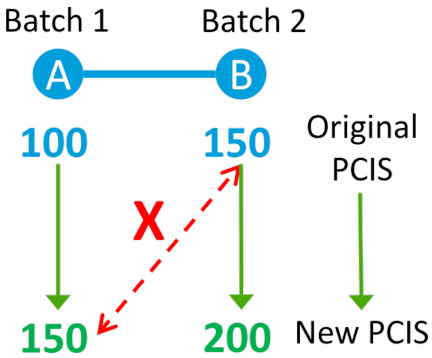
Suppose we have two neighboring cells A and B, which PCIs must be changed.

The original PCI for cell A is 100, and for cell B is 150.

Now, let us suppose that we must assign PCI 150 for cell A.

Since cells A and B are in different batches, we cannot deploy batch 1 first because it will cause a PCI collision with cell B, even though the PCI for B should be changed later on.

We call this a PCI shuffle.



Main Contributions

1. We propose a **Non-dominated Sorting BRKGA (NS-BRKGA)** to solve the **multi-objective** PCI assignment problem with conflicting objectives.
2. We applied a **random-key encoding + specialized decoder** that combines:
   - fast best-effort **constraint repair**,
   - **lexicographically guided** local search, and
   - **genotype feedback** (re-encoding improvements into chromosomes).
3. We benchmark on **291 real-world instances** against five representative MOEAs (incl. NSGA-II), showing **stronger Pareto fronts** under the same time budget.

Problem Definition

Modeling the network and neighborhoods

We model the network as an undirected graph $G = (V, E)$:

- V: cells;
- E: neighbor/interference relations.

The edge set is partitioned:

$$E = T \cup N, \qquad T \cap N = \varnothing,$$

- T: **top-neighbors** (critical relations) where modulo-k collisions are *forbidden*;
- N: regular neighbors where direct collisions are forbidden, and modulo-k collisions are undesirable.

Second-level neighbors (used to model confusions):

$$S = \{(u, w) \mid (u, v) \in E,\ (v, w) \in E,\ (u, w) \notin E\}.$$

Decision Variables and Hard Constraints

Decision variable: assign each cell $v \in V$ an integer PCI $x_v \in AP \subseteq \{0, 1, \ldots, PCI_{\max}\}$.

Hard constraints (must hold):

1. **Collision-freeness** (no adjacent equal PCIs):

$$x_u \neq x_v, \quad \forall (u, v) \in E.$$

2. **Top-neighbor modulo-k collision-freeness**:

$$x_u \not\equiv x_v \pmod{k}, \quad \forall (u, v) \in T.$$

3. **Confusion-freeness** (no two second-level neighbors share a PCI):

$$x_u \neq x_v, \quad \forall (u, v) \in S.$$

Objective Functions

Let $\mathbb{I}(\cdot)$ be the indicator function and O_v the original PCI of cell v.

1. **Modulo-k collisions on regular neighbors** (interference):

$$f_1(x) = \sum_{(u,v)\in N} \mathbb{I}\big(x_u \equiv x_v \pmod{k}\big).$$

2. **PCI shuffles** (deployment safety during batched rollouts):

$$f_2(x) = \sum_{(u,v)\in E\cup S} \Big(\mathbb{I}(x_u = O_v) + \mathbb{I}(x_v = O_u)\Big).$$

3. **Total PCI changes** (stability / renumbering cost):

$$f_3(x) = \sum_{v\in V} \mathbb{I}(x_v \neq O_v).$$

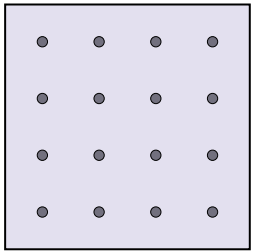
Goal: approximate the **Pareto front** for (f_1, f_2, f_3) over feasible assignments.

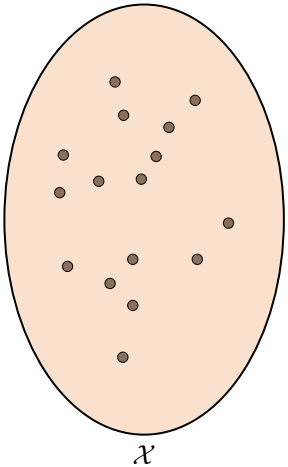
Proposed Method: NS-BRKGA

https://github.com/luishpmendes/ns-brkga

Non-dominated Sorting Biased Random-Key Genetic Algorithm

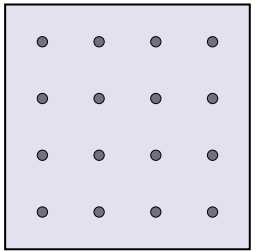
Random-Key Genetic Algorithm (RKGA)



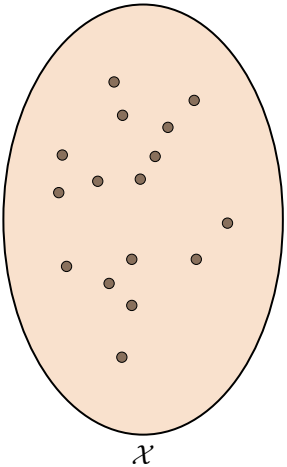


Non-dominated Sorting Biased Random-Key Genetic Algorithm

Random-Key Genetic Algorithm (RKGA)

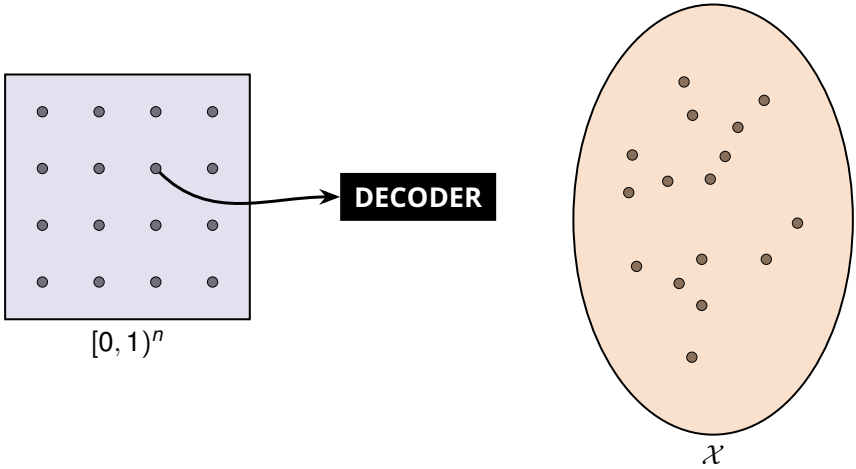


DECODER



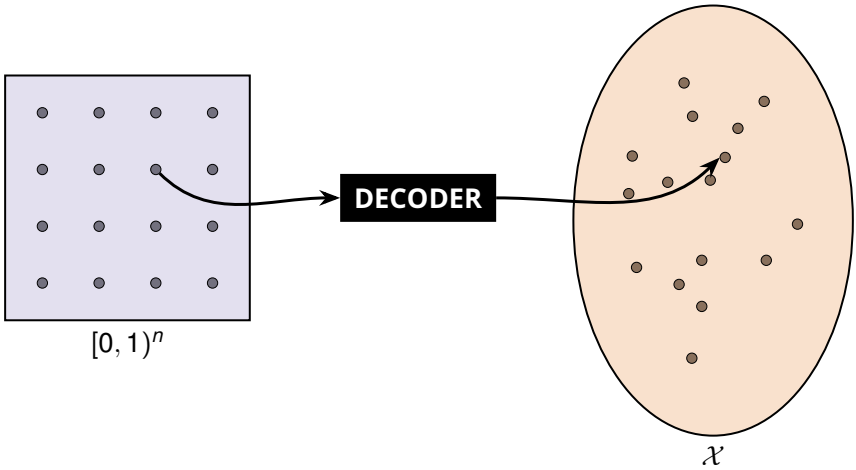
Non-dominated Sorting Biased Random-Key Genetic Algorithm

Random-Key Genetic Algorithm (RKGA)



Non-dominated Sorting Biased Random-Key Genetic Algorithm

Random-Key Genetic Algorithm (RKGA)



Non-dominated Sorting Biased Random-Key Genetic Algorithm

Solution representation for MOPCI

A chromosome is a vector of random keys:

$$\mathbf{k} = (k_v)_{v \in V}, \qquad k_v \in [0, 1).$$

Let AP be the sorted list of allowed PCIs, with $m = |AP|$.

Deterministic decoding (initial assignment):

$$x_v \;=\; AP[\lfloor k_v \cdot m \rfloor]\,.$$

- This decoding is fast, but it can yield **infeasible** assignments.
- Feasibility is enforced by an embedded repair + local improvement phase.

Non-dominated Sorting Biased Random-Key Genetic Algorithm

Decoder step 1: fast feasibility repair

We quantify hard-constraint violations via three counters:

$$v_{\text{col}}(\mathbf{x}) = \sum_{(u,v)\in E} \mathbb{I}(x_u = x_v),$$

$$v_{\text{tn}}(\mathbf{x}) = \sum_{(u,v)\in T} \mathbb{I}\big(x_u \equiv x_v \pmod{k}\big),$$

$$v_{\text{conf}}(\mathbf{x}) = \sum_{(u,v)\in S} \mathbb{I}(x_u = x_v).$$

A solution is feasible iff $v_{\text{col}} = v_{\text{tn}} = v_{\text{conf}} = 0$.

Greedy best-effort repair:

- Build the set of *violated* cells $U \subseteq V$.
- Pick $v \in U$ with the most incident violations.
- Test all $p \in AP$ and choose $x_v \leftarrow p$ that minimizes $v_{\text{col}} + v_{\text{tn}} + v_{\text{conf}}$.
- Repeat while improving the total violation count.

Non-dominated Sorting Biased Random-Key Genetic Algorithm

Decoder step 1b: penalize residual infeasibility

If repair cannot remove all violations, we penalize the first objective:

$$\tilde{f}_1(\mathbf{x}) = f_1(\mathbf{x}) + M \cdot \big(v_{\text{col}}(\mathbf{x}) + v_{\text{tn}}(\mathbf{x}) + v_{\text{conf}}(\mathbf{x})\big),$$

with a large constant M (set as $M = |N| + 1$).

Returned fitness is:

$$\big(\tilde{f}_1(\mathbf{x}),\ f_2(\mathbf{x}),\ f_3(\mathbf{x})\big).$$

- Any remaining hard-constraint violation makes $\tilde{f}_1$ worse than the worst feasible f_1 value.
- This prevents infeasible solutions from dominating feasible ones.

Non-dominated Sorting Biased Random-Key Genetic Algorithm

Decoder step 2: lexicographically guided local search

Once feasible, we run local search over **feasible single-cell moves**:

$$x_v \leftarrow p, \quad p \in AP \setminus \{x_v\} \text{ and move preserves feasibility.}$$

Move acceptance (inside the decoder):
first-improvement under a lexicographic order

$$(a_1, a_2, a_3) \prec_{\text{lex}} (b_1, b_2, b_3) \Leftrightarrow \begin{cases} a_1 < b_1, \text{ or} \\ a_1 = b_1 \ \wedge \ a_2 < b_2, \text{ or} \\ a_1 = b_1 \ \wedge \ a_2 = b_2 \ \wedge \ a_3 < b_3. \end{cases}$$

- Local search prioritizes operational objectives in the order (f_1, f_2, f_3).
- Example: increasing f_3 is accepted only if it improves f_2 or f_1.

Non-dominated Sorting Biased Random-Key Genetic Algorithm

Decoder step 3: genotype feedback

After local search improves **x**, we **re-encode** the solution back into the chromosome.

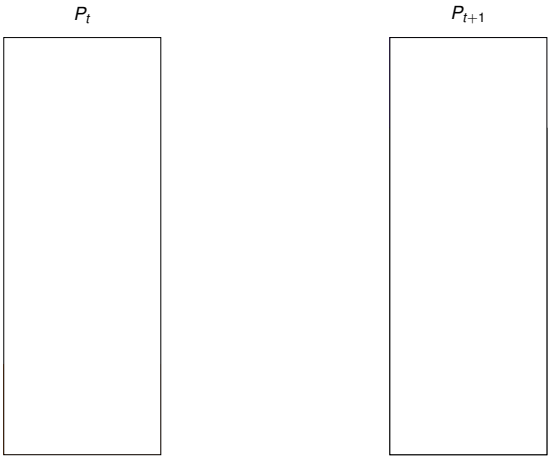
If $x_v = AP[j]$ for $j \in \{0, \ldots, |AP| - 1\}$, set:

$$k_v \leftarrow \frac{j + 0.5}{|AP|} \quad \Rightarrow \quad \lfloor k_v \cdot |AP| \rfloor = j.$$

- The decoder becomes a **memetic operator**: improvements are not lost.
- Evolutionary operators can now recombine and propagate locally improved building blocks.

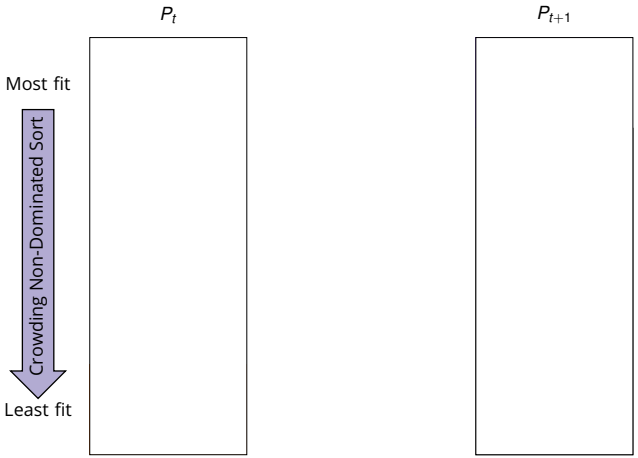
Non-dominated Sorting Biased Random-Key Genetic Algorithm

Evolution



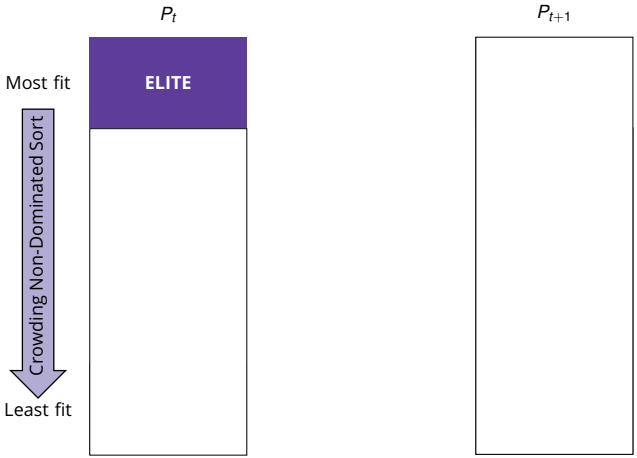
Non-dominated Sorting Biased Random-Key Genetic Algorithm

Evolution



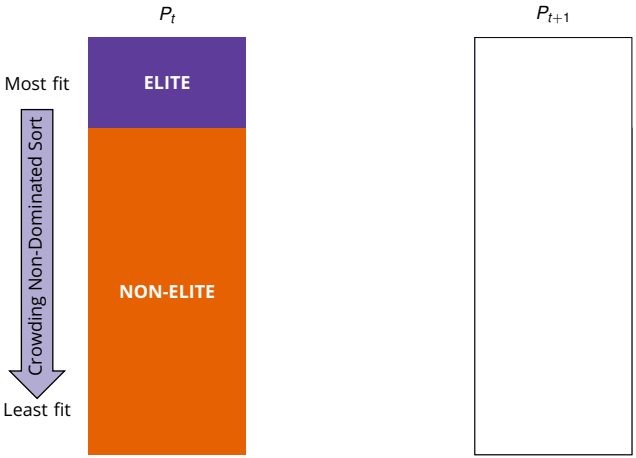
Non-dominated Sorting Biased Random-Key Genetic Algorithm

Evolution



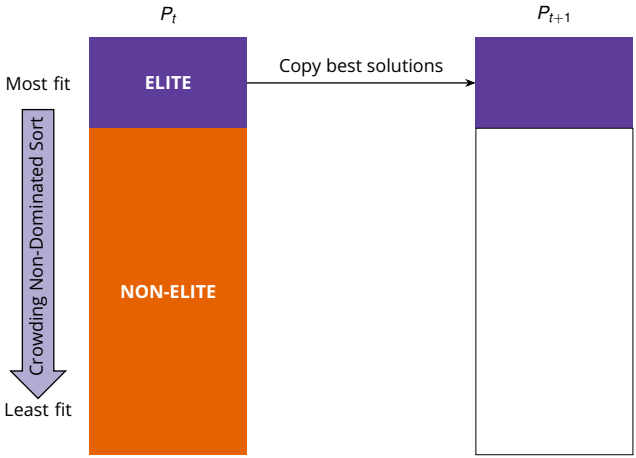
Non-dominated Sorting Biased Random-Key Genetic Algorithm

Evolution



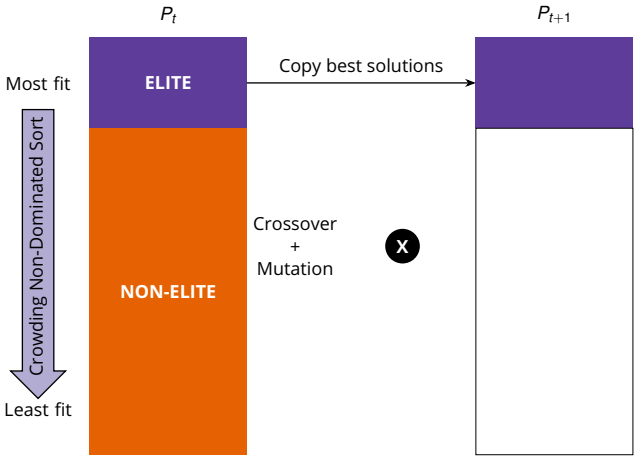
Non-dominated Sorting Biased Random-Key Genetic Algorithm

Evolution



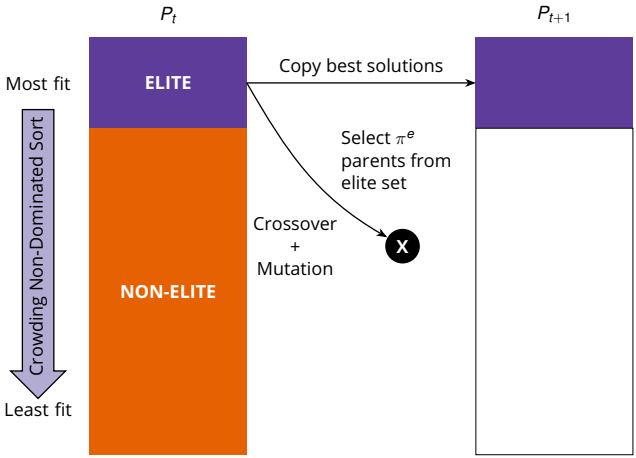
Non-dominated Sorting Biased Random-Key Genetic Algorithm

Evolution



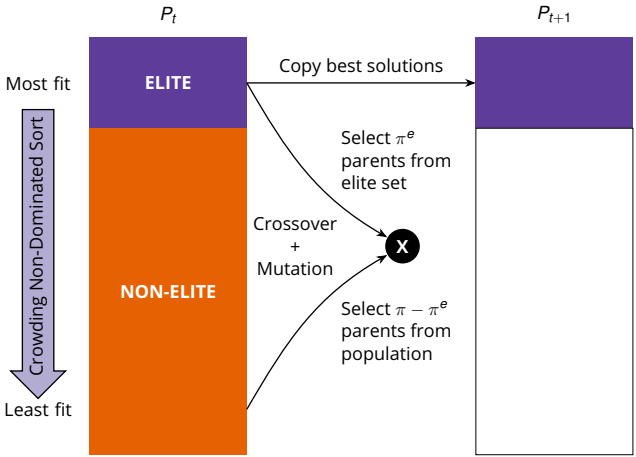
Non-dominated Sorting Biased Random-Key Genetic Algorithm

Evolution



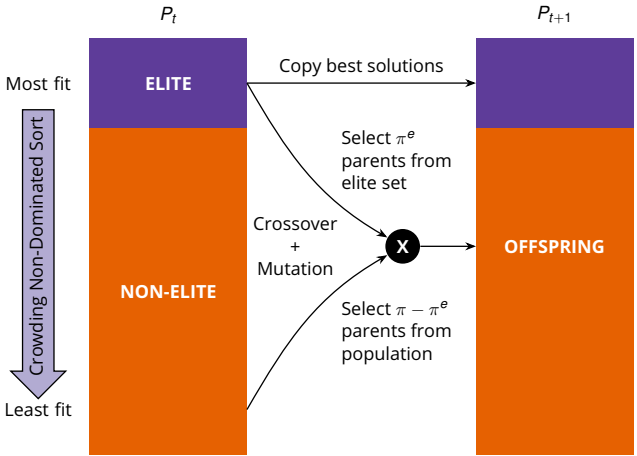
Non-dominated Sorting Biased Random-Key Genetic Algorithm

Evolution



Non-dominated Sorting Biased Random-Key Genetic Algorithm

Evolution



Non-dominated Sorting Biased Random-Key Genetic Algorithm

Features

- Biased Multi-Parent Crossover.
- Polynomial Mutation.
- Genotypic Diversity at Elite Selection.
- Migration on Multiple Populations.
- Implicit Path-Relinking.
- Shaking.
- Partial reset.

Computational Experiments

Experimental Setup

- 291 instances.
- 5 independent runs for each algorithm and instance.
- 15 minutes per run.
- Same reference fronts and indicators.
- Same implementation language (C++).

Computer setup: CPU of 3.30GHz and 32GB of RAM.

Benchmark Instances

- 291 real-world MOPCI instances (sizes from 30 to 4125 cells)[1].
- Instance types (PCI groups):
  - `full`: $AP = \{0, \ldots, 1007\}$ (least constrained),
  - `orig`: $AP = \{\min(\mathbf{O}), \ldots, \max(\mathbf{O})\}$,
  - `squz`: minimal contiguous AP that still admits feasibility (most constrained).

[1] Available at https://github.com/ceandrade/pci_assignment_problem_instances

Baseline Algorithms

The proposed NS-BRKGA is benchmarked against the following algorithms[2]:

- **NSGA-II**: The Elitist Non-dominated Sorting Genetic Algorithm.
- **NSPSO**: The Non-dominated Sorting Particle Swarm Optimizer.
- **MOEA/D-DE**: The Multi-Objective Evolutionary Algorithm by Decomposition with Differential Evolution.
- **MHACO**: The Multi-objective Hypervolume-based Ant Colony Optimizer.
- **IHS**: The Improved Harmony Search.

[2] Implementations at: https://esa.github.io/pagmo2/

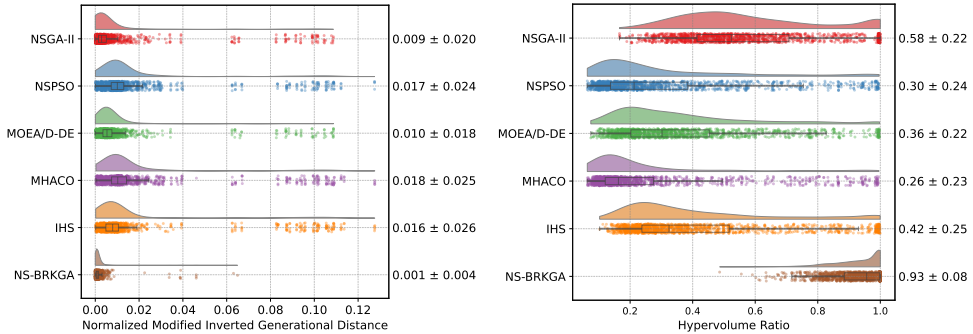
Performance Indicators

Two complementary quality indicators were employed:

1. **Normalized Modified Inverted Generational Distance** ($NIGD^{+}$).
   - lower is better
2. **Hypervolume Ratio** (HVR).
   - higher is better

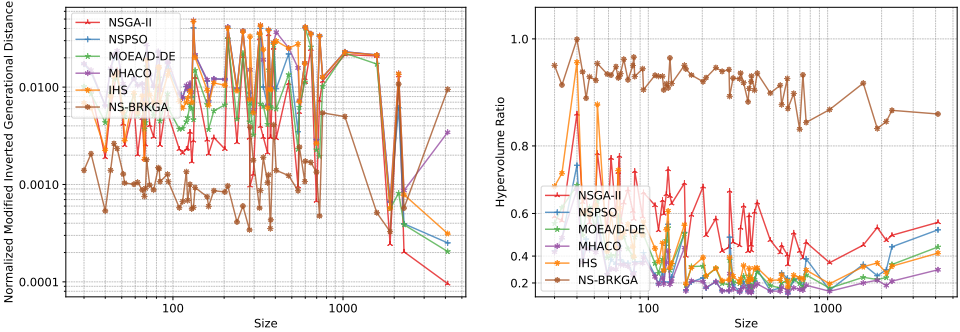
Results & Analysis

Overall Performance Comparison



- NS-BRKGA attains the best mean ranks on **both** $NIGD^+$ and HVR.
- Reported means: $NIGD^+ \approx 0.001$ and $HVR \approx 0.93$ (NSGA-II: 0.009 and 0.58).

Scalability with Problem Size



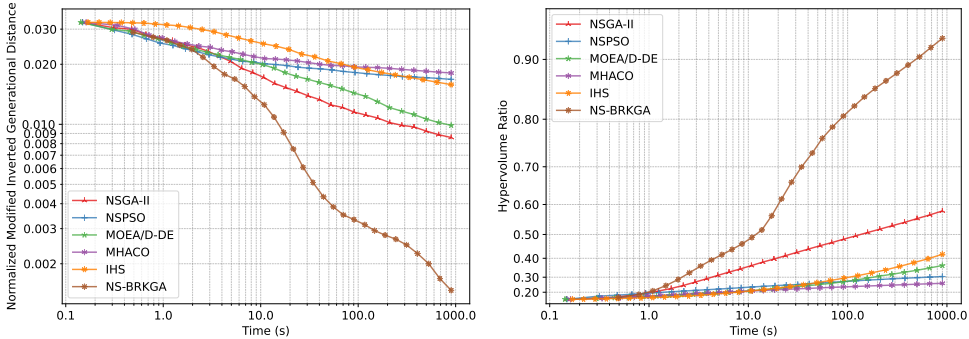
- As $|V|$ grows, baseline methods degrade, while NS-BRKGA remains stable.
- On the largest instances ($|V| = 4125$), NS-BRKGA maintains mean $HVR \approx 0.87 \pm 0.08$ (NSGA-II $\approx 0.6 \pm 0.1$).

Robustness Across Instance Types

Algorithm	*NIGD*$^+$ (↓)			*HVR* (↑)		
	`full`	`orig`	`squz`	`full`	`orig`	`squz`
NSGA-II	0.005 ± 0.006	0.004 ± 0.004	0.017 ± 0.032	0.46 ± 0.17	0.51 ± 0.11	0.77 ± 0.23
NSPSO	0.011 ± 0.007	0.011 ± 0.006	0.029 ± 0.037	0.22 ± 0.14	0.20 ± 0.11	0.49 ± 0.30
MOEA/D-DE	0.007 ± 0.005	0.006 ± 0.004	0.017 ± 0.030	0.27 ± 0.12	0.30 ± 0.13	0.53 ± 0.27
MHACO	0.011 ± 0.007	0.011 ± 0.007	0.033 ± 0.039	0.15 ± 0.07	0.17 ± 0.09	0.47 ± 0.30
IHS	0.008 ± 0.005	0.007 ± 0.004	0.033 ± 0.040	0.33 ± 0.17	0.36 ± 0.16	0.57 ± 0.30
NS-BRKGA	$\mathbf{0.001 \pm 0.001}$	$\mathbf{0.001 \pm 0.001}$	$\mathbf{0.002 \pm 0.007}$	$\mathbf{0.90 \pm 0.09}$	$\mathbf{0.91 \pm 0.08}$	$\mathbf{0.97 \pm 0.05}$

- NS-BRKGA is the top performer on all instance types.
- On `squz`: mean *HVR* ≈ 0.97 and mean *NIGD*$^+$ ≈ 0.002.

Convergence & Runtime Performance



- NS-BRKGA reaches high-quality fronts **very early** (within $\sim$ 30s: $HVR \approx 0.7 \pm 0.2$).
- Baselines often do not match this quality even after the full 15-minute budget.

Conclusion

Conclusion

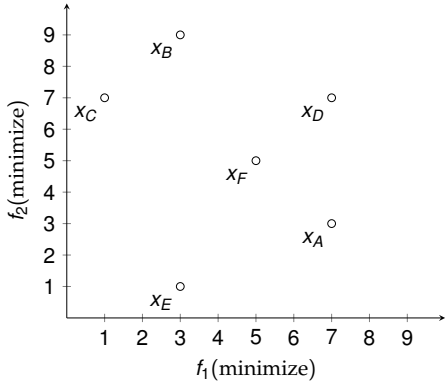
- **MOPCI** is a practical LTE/5G optimization problem involving both **technological constraints** and multi-objective **operational concerns**.
- **Methodological contribution:** NS-BRKGA integrates a random-key evolutionary framework with a problem-specific decoder featuring **feasibility repair**, **lexicographically guided local search**, and **genotype feedback**.
- **Experimental evidence:** Across 291 real-world instances and five representative MOEAs, NS-BRKGA consistently achieves **strong Pareto-front quality** ($HVR \approx 0.93$, $NIGD^{+} \approx 0.001$), **scalability**, **robustness**, and **fast convergence** ($\sim$ 30s).
- **Take-home message:** NS-BRKGA provides network operators with a highly practical way to quickly generate high-quality PCI plans that balance **radio performance**, **rollout safety**, and **operational stability**.

That’s all, folks!

Backup Slides

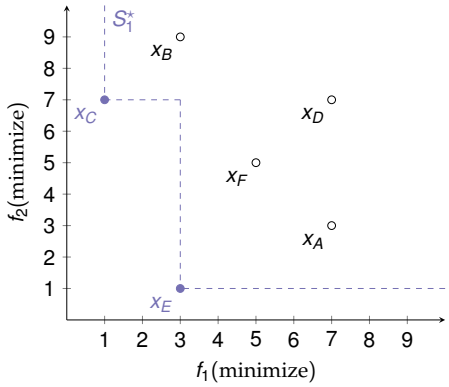
Non-dominated Sorting Biased Random-Key Genetic Algorithm

Non-dominated Sort



Non-dominated Sorting Biased Random-Key Genetic Algorithm

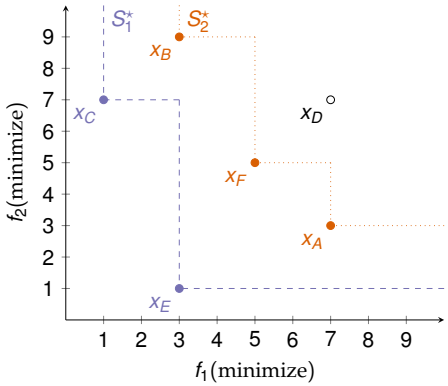
Non-dominated Sort



$$S_1^\star = \{x_C, x_E\}$$

Non-dominated Sorting Biased Random-Key Genetic Algorithm

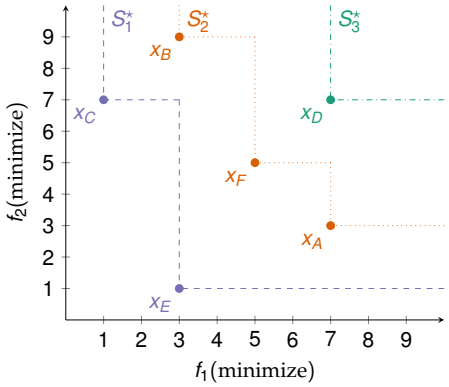
Non-dominated Sort



$S_1^\star = \{x_C, x_E\}$ $\quad$ $S_2^\star = \{x_A, x_B, x_F\}$

Non-dominated Sorting Biased Random-Key Genetic Algorithm

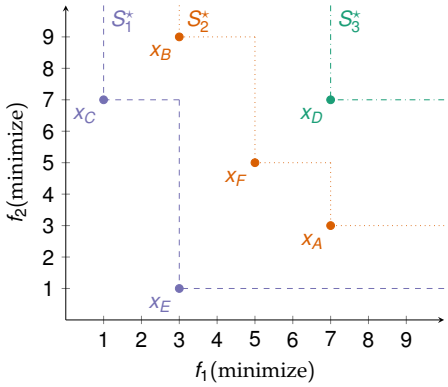
Non-dominated Sort



$S_1^\star = \{x_C, x_E\}$ $S_2^\star = \{x_A, x_B, x_F\}$ $S_3^\star = \{x_D\}$

Non-dominated Sorting Biased Random-Key Genetic Algorithm

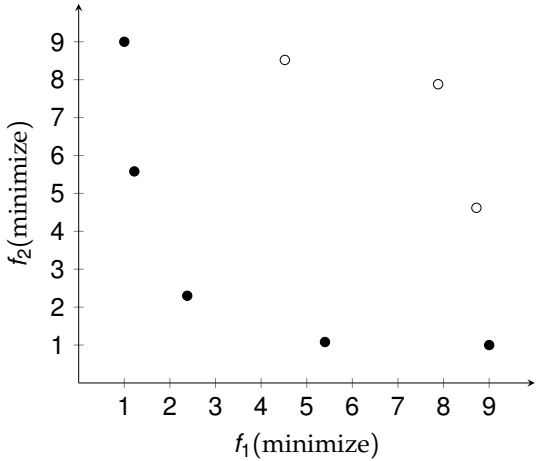
Non-dominated Sort



$$S_1^\star = \{x_C, x_E\} \prec S_2^\star = \{x_A, x_B, x_F\} \prec S_3^\star = \{x_D\}$$

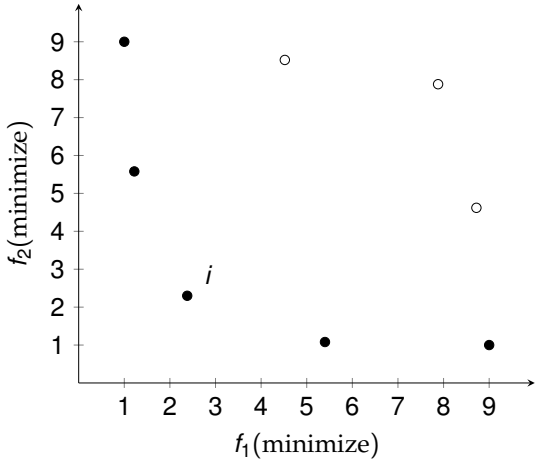
Non-dominated Sorting Biased Random-Key Genetic Algorithm

Crowding Distance



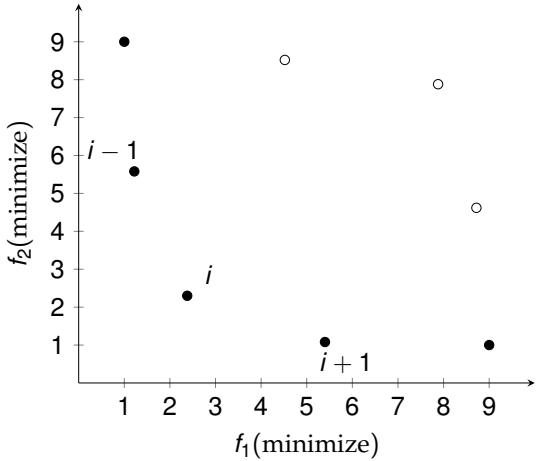
Non-dominated Sorting Biased Random-Key Genetic Algorithm

Crowding Distance



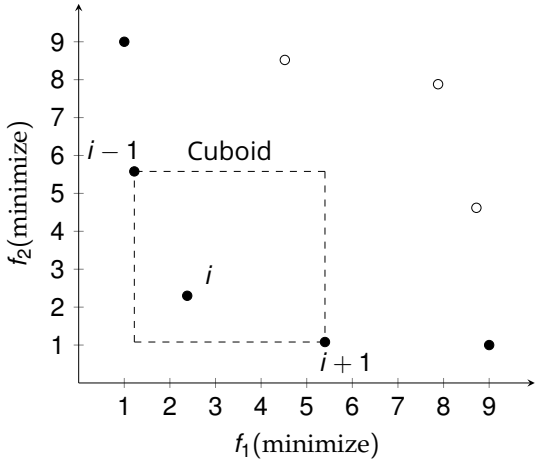
Non-dominated Sorting Biased Random-Key Genetic Algorithm

Crowding Distance



Non-dominated Sorting Biased Random-Key Genetic Algorithm

Crowding Distance



Non-dominated Sorting Biased Random-Key Genetic Algorithm

Multi-Parent Biased Crossover

Non-dominated Sorting Biased Random-Key Genetic Algorithm

Multi-Parent Biased Crossover

Parent 1	0.16	0.53	0.85	0.76	0.99	0.06	0.38
Parent 2	0.12	0.63	0.21	0.69	0.08	0.01	0.90

03

05

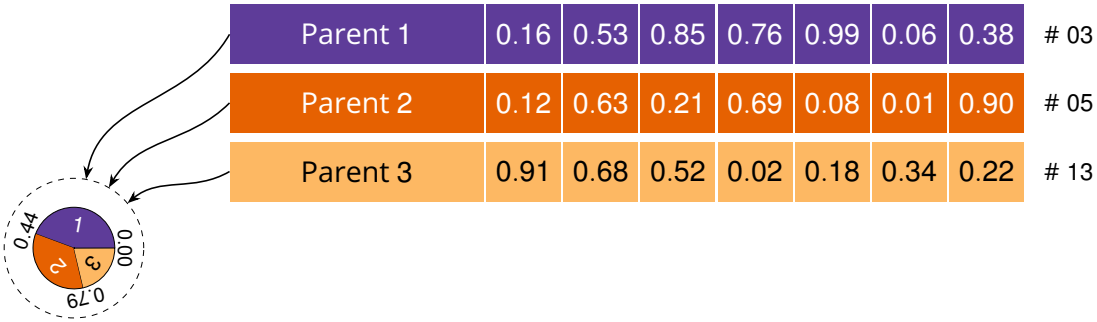
Non-dominated Sorting Biased Random-Key Genetic Algorithm

Multi-Parent Biased Crossover

Parent 1	0.16	0.53	0.85	0.76	0.99	0.06	0.38	# 03
Parent 2	0.12	0.63	0.21	0.69	0.08	0.01	0.90	# 05
Parent 3	0.91	0.68	0.52	0.02	0.18	0.34	0.22	# 13

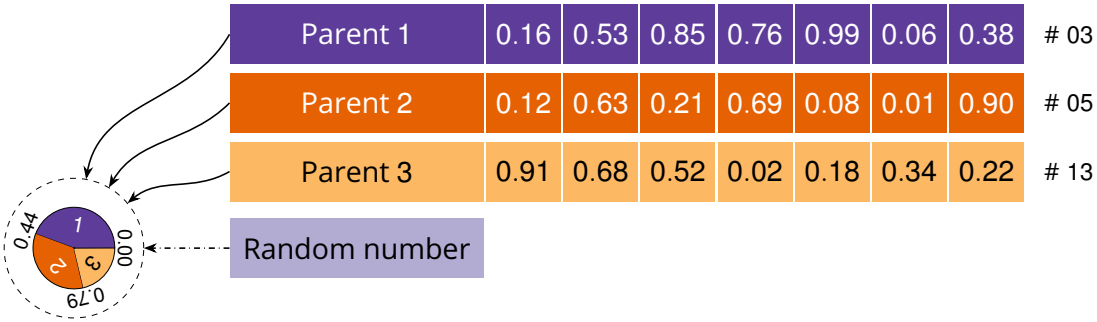
Non-dominated Sorting Biased Random-Key Genetic Algorithm

Multi-Parent Biased Crossover



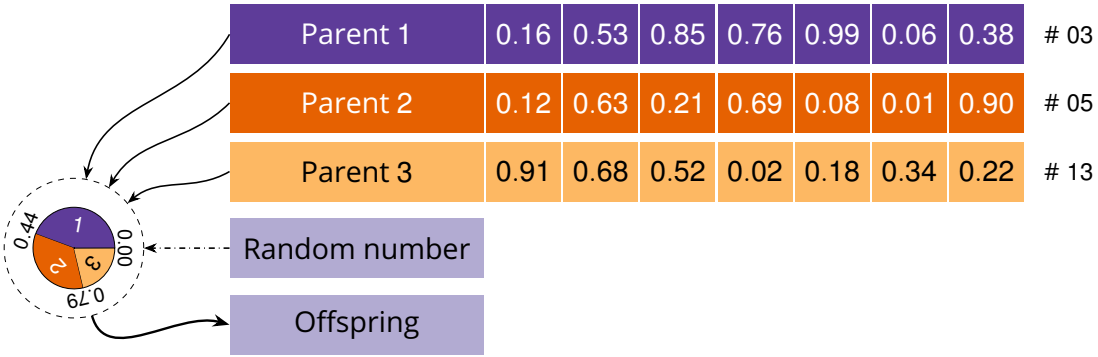
Non-dominated Sorting Biased Random-Key Genetic Algorithm

Multi-Parent Biased Crossover



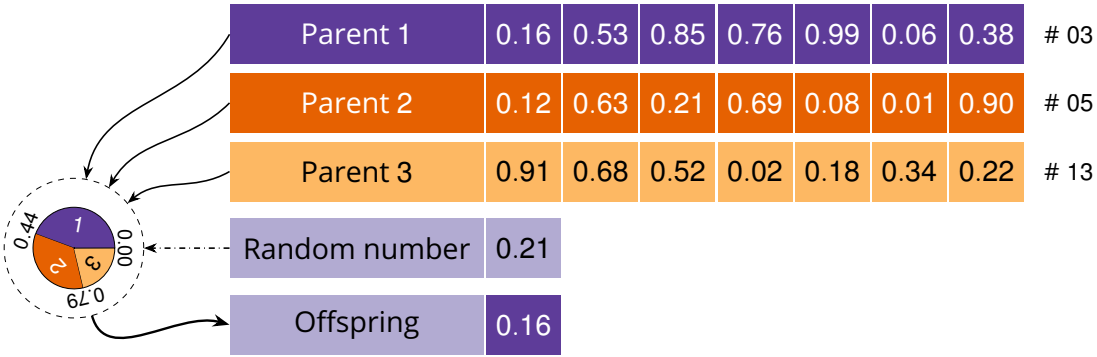
Non-dominated Sorting Biased Random-Key Genetic Algorithm

Multi-Parent Biased Crossover



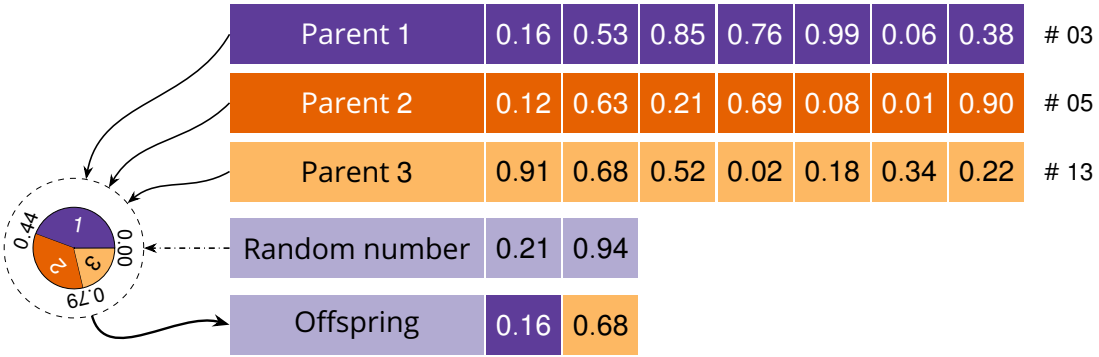
Non-dominated Sorting Biased Random-Key Genetic Algorithm

Multi-Parent Biased Crossover



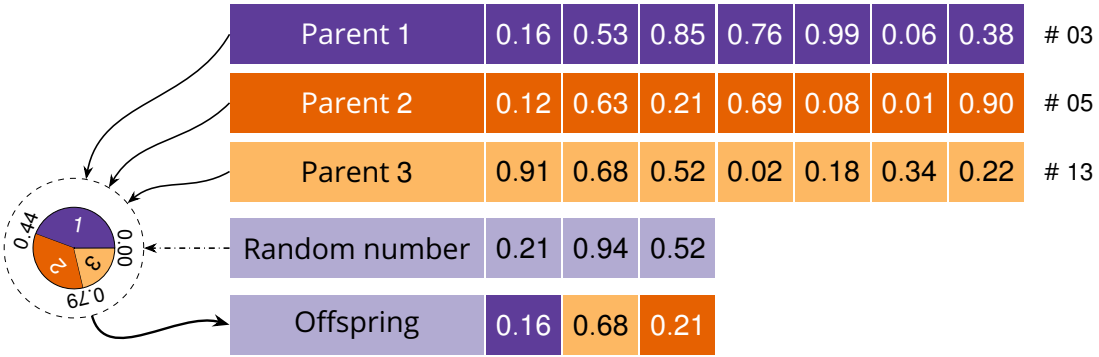
Non-dominated Sorting Biased Random-Key Genetic Algorithm

Multi-Parent Biased Crossover



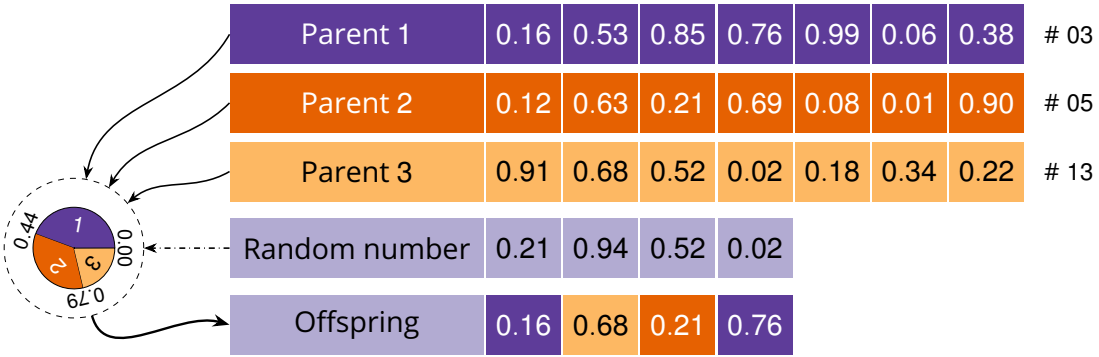
Non-dominated Sorting Biased Random-Key Genetic Algorithm

Multi-Parent Biased Crossover



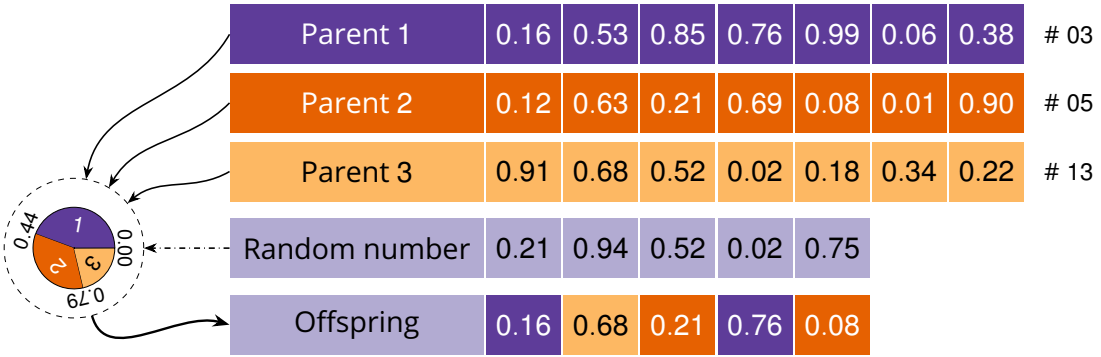
Non-dominated Sorting Biased Random-Key Genetic Algorithm

Multi-Parent Biased Crossover



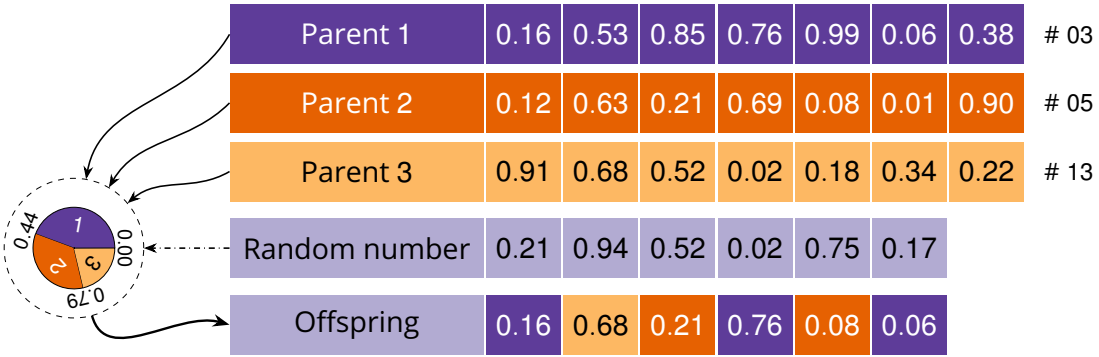
Non-dominated Sorting Biased Random-Key Genetic Algorithm

Multi-Parent Biased Crossover



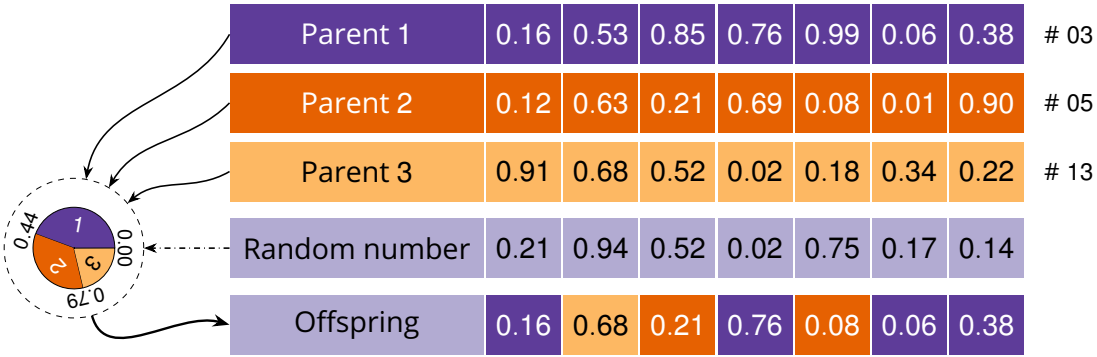
Non-dominated Sorting Biased Random-Key Genetic Algorithm

Multi-Parent Biased Crossover



Non-dominated Sorting Biased Random-Key Genetic Algorithm

Multi-Parent Biased Crossover



Non-dominated Sorting Biased Random-Key Genetic Algorithm

Polynomial Mutation

Applied with probability p_m to induce an expected perturbation of $\mathcal{O}(\frac{1}{\eta_m})$ in a gene.

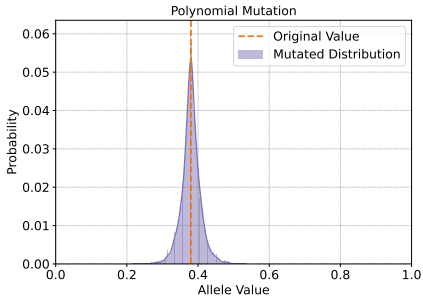
Generates a new allele as follows:

$$k'_v = k_v + \delta,$$

where δ is a perturbation value calculated from a random number $u \in [0, 1)$, as follows:

$$\delta = \begin{cases} (2u + (1-2u)(1-k_v)^{\eta_m+1})^{\frac{1}{\eta_m+1}} - 1 & \text{if } u < \frac{1}{2}, \\ 1 - (2(1-u) + 2(u-\frac{1}{2})(k_v)^{\eta_m+1})^{\frac{1}{\eta_m+1}} & \text{otherwise.} \end{cases}$$

$$k_v = 0.38 \Rightarrow \quad k'_v \approx 0.38 \pm 0.03$$
$$\eta_m = 50.00 \Rightarrow \quad |k'_v - k_v| \approx 0.02 \pm 0.02$$



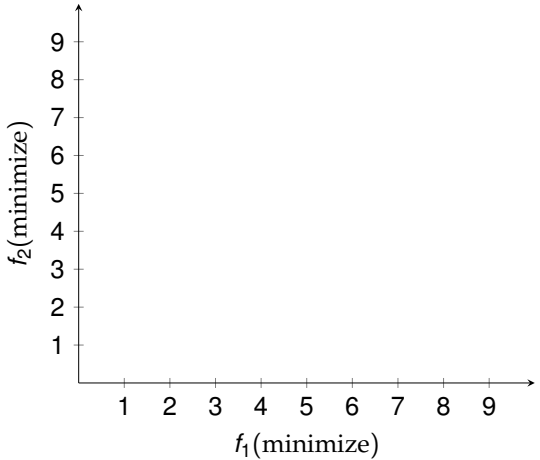
Experimental Setup

NS-BRKGA parameter settings

Parameter	**Value**
Population size $\|P\|$ (per subpopulation)	300
Parent count π (elitist π_e)	3 (with $\pi_e = 2$ elites)
Elite fraction $p^e_{\min}$ – $p^e_{\max}$	0.10 – 0.30
Bias function $\Phi(r)$	$1/\sqrt{r}$
Mutation probability p_m (index η_m)	0.01 ($\eta_m = 50$)
Number of subpopulations p	3
Migration interval (gen), size k	200 gen, $k = 30$ solutions
IPR interval (gen)	500 gen
Shaking trigger (stagnant gen)	200 gen ($p_{\text{shake}} = 0.33$, $\eta_{\text{shake}} = 20$)
Reset trigger (stagnant gen)	500 gen ($p_{\text{reset}} = 0.20$)

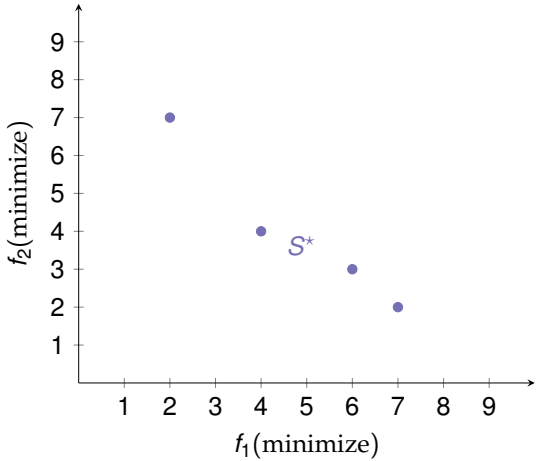
Performance Indicators

Normalized Modified Inverted Generational Distance ($NIGD^+$)



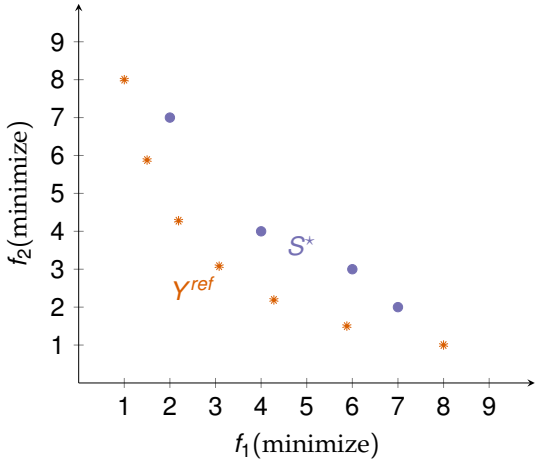
Performance Indicators

Normalized Modified Inverted Generational Distance ($NIGD^+$)



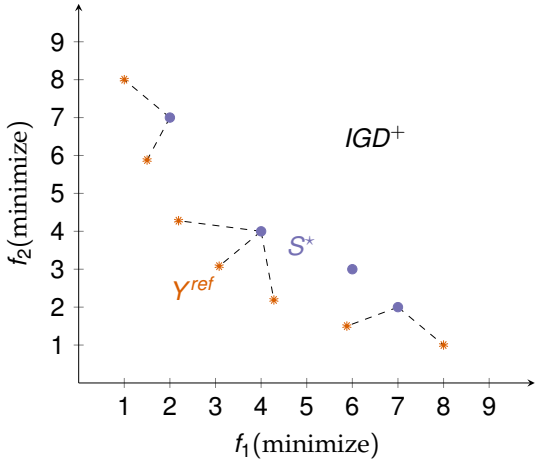
Performance Indicators

Normalized Modified Inverted Generational Distance ($NIGD^+$)



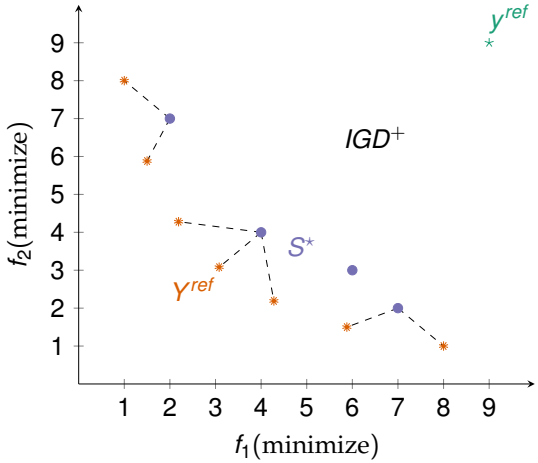
Performance Indicators

Normalized Modified Inverted Generational Distance ($NIGD^+$)



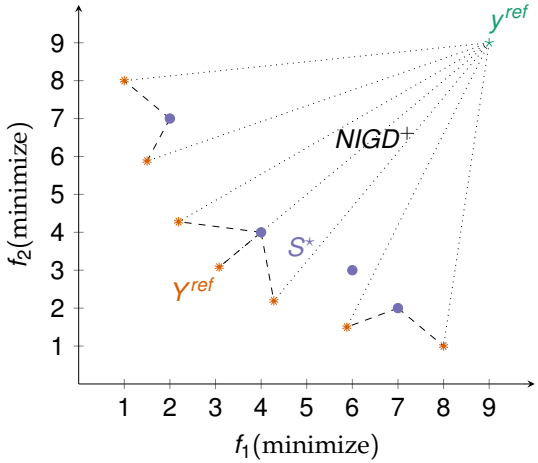
Performance Indicators

Normalized Modified Inverted Generational Distance ($NIGD^+$)



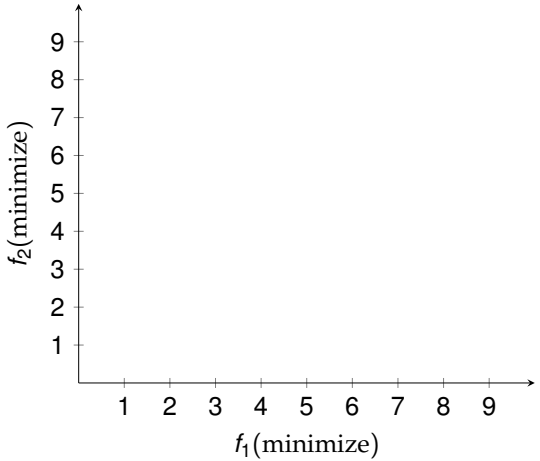
Performance Indicators

Normalized Modified Inverted Generational Distance ($NIGD^+$)



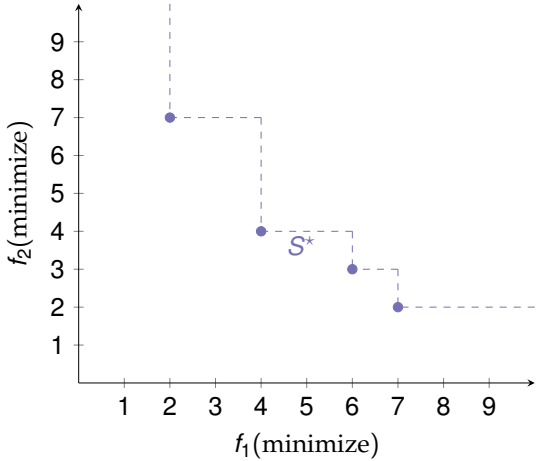
Performance Indicators

Hypervolume Ratio (*HVR*)



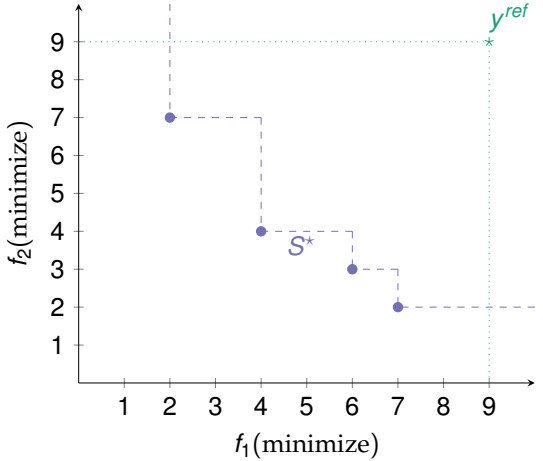
Performance Indicators

Hypervolume Ratio (*HVR*)



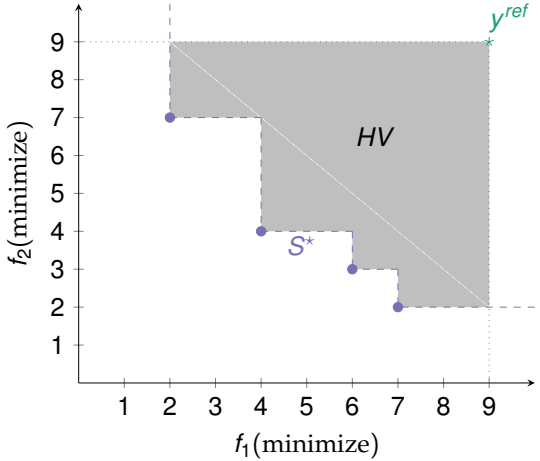
Performance Indicators

Hypervolume Ratio (*HVR*)



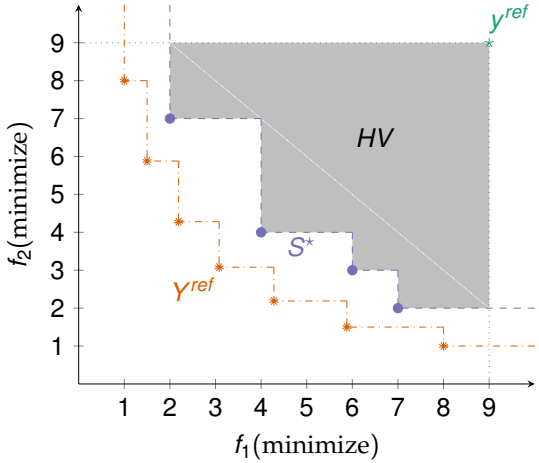
Performance Indicators

Hypervolume Ratio (*HVR*)



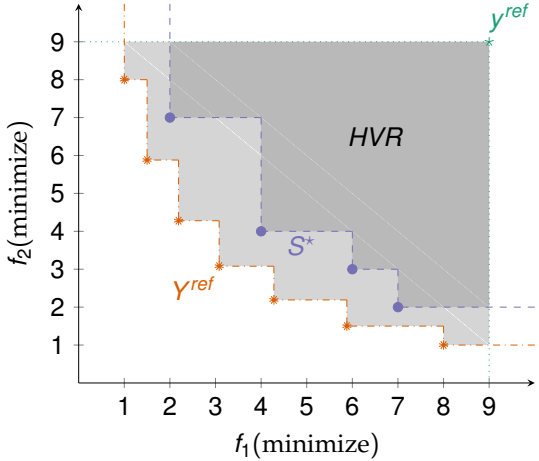
Performance Indicators

Hypervolume Ratio (*HVR*)



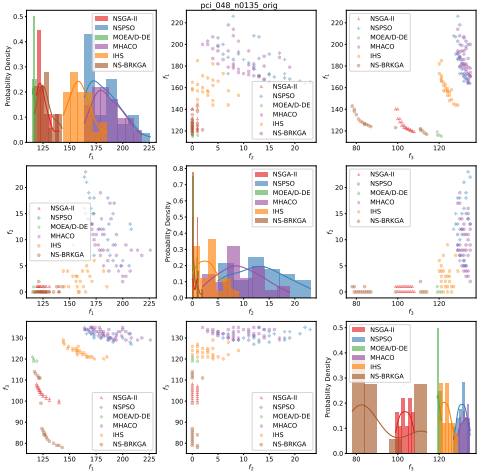
Performance Indicators

Hypervolume Ratio (*HVR*)

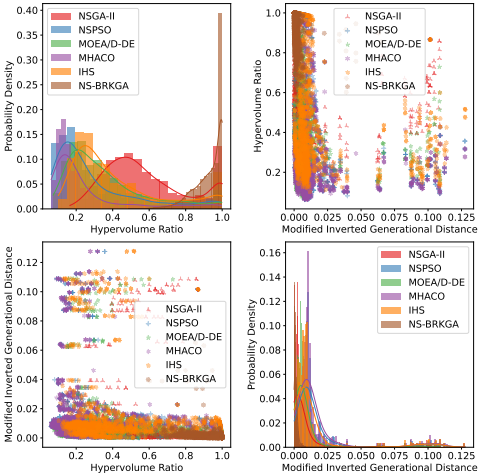


Results for a single instance

pci_048_n0135_orig_best



Overall Performance Comparison



Robustness Across Instance Types